

# Day 6 Role-Based Authorization

## Date

August 26, 2026

## Overview

Implemented role-based access control (RBAC) using Spring Security's `@PreAuthorize` annotations and Thymeleaf security tags. Three roles: ADMIN, DOCTOR, RECEPTIONIST with different permissions for Patients, Doctors, and UI elements.

## Tech Stack

- Java 26
- Spring Boot 4.1.0
- Lombok 1.18.46
- MongoDB (clinic\_db)
- Thymeleaf + Bootstrap 5 + Thymeleaf-SpringSecurity6
- Spring Security with `@EnableMethodSecurity`
- JWT for REST API + Session for Browser

## Files Created / Updated

### New Java Files

1. `com.clinic.controller.DebugController` - Diagnostic endpoint for auth state

### Updated Java Files

1. `com.clinic.config.SecurityConfig` - Added `@EnableMethodSecurity(prePostEnabled = true)`
2. `com.clinic.controller.PatientRestController` - `@PreAuthorize` on all endpoints
3. `com.clinic.controller.PatientViewController` - `@PreAuthorize` on create/edit/delete
4. `com.clinic.controller.DoctorRestController` - `@PreAuthorize` on all endpoints
5. `com.clinic.controller.DoctorViewController` - `@PreAuthorize` on create/edit/delete
6. `com.clinic.security.JwtAuthenticationFilter` - Skip filter for non-API paths using `shouldNotFilter()`
7. `com.clinic.security.CustomUserDetailsService` - Explicit `ROLE_` prefix with `SimpleGrantedAuthority`

### Updated HTML Files

1. `templates/auth/login.html` - Form posts to Spring Security `/login` (not custom controller)
2. `templates/index.html` - `sec:authorize` for Users card, login/logout nav, debug info
3. `templates/patients/list.html` - Hide Add/Edit/Delete buttons by role
4. `templates/patients/detail.html` - Hide Edit/Delete buttons by role
5. `templates/doctors/list.html` - Hide Add/Edit/Delete buttons by role
6. `templates/doctors/detail.html` - Hide Edit/Delete buttons by role

## Role-Based Access Matrix

| Action          | ADMIN | DOCTOR | RECEPTIONIST |
|-----------------|-------|--------|--------------|
| View Patients   | ✓     | ✓      | ✗            |
| Add Patient     | ✓     | ✗      | ✗            |
| Edit Patient    | ✓     | ✗      | ✓            |
| Delete Patient  | ✓     | ✗      | ✗            |
| View Doctors    | ✓     | ✓      | ✓            |
| Add Doctor      | ✓     | ✗      | ✗            |
| Edit Doctor     | ✓     | ✗      | ✗            |
| Delete Doctor   | ✓     | ✗      | ✗            |
| View Users Card | ✓     | ✗      | ✗            |

## REST API Endpoints with Roles

| Method | Endpoint           | Allowed Roles               |
|--------|--------------------|-----------------------------|
| GET    | /api/patients      | ADMIN, DOCTOR, RECEPTIONIST |
| GET    | /api/patients/{id} | ADMIN, DOCTOR, RECEPTIONIST |
| POST   | /api/patients      | ADMIN, RECEPTIONIST         |
| PUT    | /api/patients/{id} | ADMIN, RECEPTIONIST         |
| DELETE | /api/patients/{id} | ADMIN                       |
| GET    | /api/doctors       | ADMIN, DOCTOR, RECEPTIONIST |
| GET    | /api/doctors/{id}  | ADMIN, DOCTOR, RECEPTIONIST |
| POST   | /api/doctors       | ADMIN                       |
| PUT    | /api/doctors/{id}  | ADMIN                       |
| DELETE | /api/doctors/{id}  | ADMIN                       |

## Key Features Implemented

1. @PreAuthorize annotations on REST controllers
2. @PreAuthorize annotations on View controllers
3. Thymeleaf sec:authorize tags to conditionally render UI elements
4. Login/logout navigation based on authentication state
5. Username display in navbar when logged in
6. Dual authentication: JWT for API, Session for Browser
7. JwtAuthenticationFilter skips non-API paths using shouldNotFilter()

## Issues Faced & Solutions

### Issue 1: Thymeleaf sec:authorize showing “NOT authenticated”

- Root cause: SessionCreationPolicy.STATELESS killed form login sessions
  - Solution: Removed STATELESS, let Spring Security manage sessions normally
- Issue 2: JwtAuthenticationFilter interfering with session auth**

- Root cause: Filter ran on every request, overwriting SecurityContext
- Solution: shouldNotFilter() to skip filter for non-API paths entirely

### Issue 3: Login form posting to custom controller instead of Spring Security

- Root cause: Form action was /auth/login (custom controller), not /login (Spring Security)
- Solution: Changed form action to /login with name=“username” and name=“password” inputs

### Issue 4: hasRole vs hasAuthority mismatch

- Root cause: CustomUserDetailsService used .roles() which prepends ROLE\_
- Solution: Explicit SimpleGrantedAuthority(“ROLE\_” + role.name())

## Testing

### Browser Tests

- Login as ADMIN → All buttons visible, Users card shown, navbar shows username
- Login as DOCTOR → View only, no Add/Edit/Delete buttons
- Login as RECEPTIONIST → Can manage patients, view doctors only

### API Tests

- POST /api/auth/login → Returns JWT token
- GET /api/auth/me with Bearer token → Returns user data
- GET /api/auth/me without token → 401 Unauthorized
- DOCTOR token calling DELETE /api/patients/{id} → 403 Forbidden
- RECEPTIONIST token calling POST /api/doctors → 403 Forbidden

## Git Commit

feat: add role-based authorization with @PreAuthorize and Thymeleaf security tags

•